

```
create mode 100644 deleteme.txt
C:\Users\Al\wizcoin>git status
On branch master
nothing to commit, working tree clean
```

Don't delete the file using the `del` command on Windows or `rm` command on macOS and Linux. (If you do, you can run `git restore <filename>` to recover it or simply continue with the `git rm` command to remove it from the repo.) Instead, use the `git rm` command to delete and stage the *deleteme.txt* file such as in this example:

```
C:\Users\Al\wizcoin>git rm deleteme.txt
rm deleteme.txt'
```

The `git rm` command deletes the file from your working copy, but you're not done yet. Like `git add`, the `git rm` command stages the file. You need to commit file deletion just like any other change:

```
C:\Users\Al\wizcoin>git status
On branch master
Changes to be committed:
  ❶ (use "git reset HEAD <file>..." to unstage)

    deleted:    deleteme.txt

C:\Users\Al\wizcoin>git commit -m "Deleting deleteme.txt from the repo to finish the deletion test."
[master 369de78] Deleting deleteme.txt from the repo to finish the deletion test.
 1 file changed, 1 deletion(-)
 delete mode 100644 deleteme.txt
C:\Users\Al\Desktop\wizcoin>git status
On branch master
nothing to commit, working tree clean
```

Even though you've deleted *deleteme.txt* from your working copy, it still exists in the repo's history. The "Recovering Old Changes" section later in this chapter describes how to recover a deleted file or undo a change.

The `git rm` command only works on files that are in the clean, committed state, without any modifications. Otherwise, Git asks you to commit the changes or revert them with the `git reset HEAD <filename>` command. (The